MoE Dynamic Load-Balancing

LauzHack × Huawei CPL Zurich — Final Submission

Author: Hassan · Decoupled-Std Replication + Makespan Refinement

Four innovations:

① Dual-EMA (mean & std) ② Decoupled replication hedge (λ_rep=10) ③ Makespan local-search refinement ④ Cost-gated layer selection

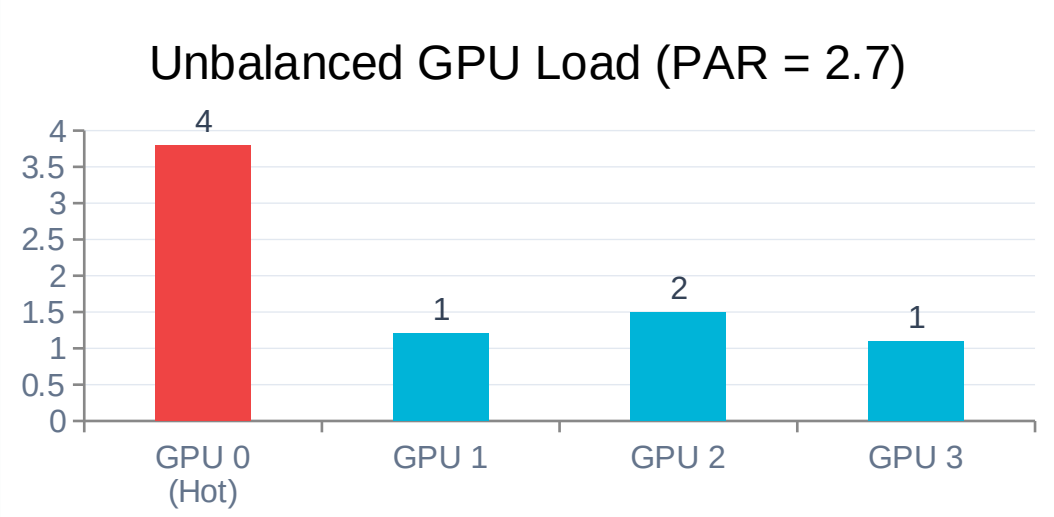
The Problem: MoE Expert Load Imbalance

Why expert placement is the critical lever at serving time

In DeepSeek-style MoE models (256 routed experts, top-8 per token), tokens are non-uniformly routed — hot experts create GPU bottlenecks. The competition focuses on serving-time placement: decide which experts to replicate and where, to minimise PAR without excessive transfer cost.

```
PAR = max(loads) / mean(loads)

lower is better · PAR=1 is balanced
```



Expert Hotness
Some experts attract 3-5× more tokens than average — driven by domain, language, token pattern.

Replication Lever
Extra GPU slots (n_red) hold replicas of hot experts, dividing their load across devices.

Transfer Cost
Moving experts between GPUs has a bandwidth cost. Score penalises unnecessary transfers.

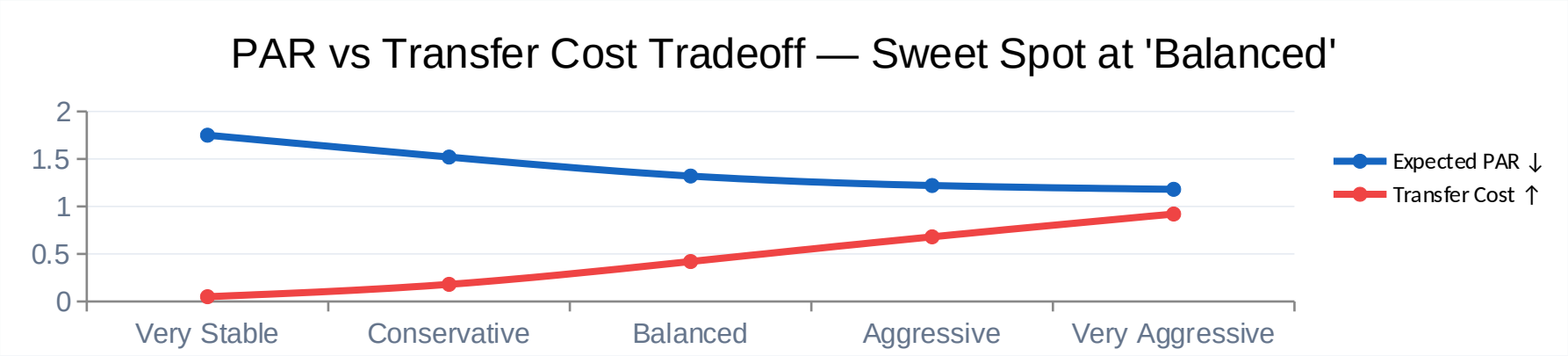
The Scoring Function — What We're Optimising

Composite score balances GPU efficiency vs. transfer bandwidth

```
modeled_time  =  compute_time  +  transfer_time

      compute_time   =  60s × mean_PAR
   transfer_time   =  transmit × 88 MB / 900 GB/s
```

Score = 100 × modeled_time / baseline_modeled_time → lower is better, algorithm must run within 3× baseline time



Baseline vs. Final Submission

DS-EPLB → Decoupled-Std Replication + Makespan Refinement

Baseline: DS-EPLB

- ✗ Raw per-window hotness (no temporal smoothing)
- ✗ Same weight for placement and replication
- ✗ LPT bin-packing only (4/3 makespan gap)
- ✗ No cost-gating: redeploys regardless of benefit
- ✗ Sensitive to out-of-window expert spikes

VS

Final: Adaptive Robust Placement

- ✓ Dual-EMA on mean & std (α=0.4) smooths noise
- ✓ Decoupled replication: heavier std hedge (λ_rep=10)
- ✓ Makespan local-search closes the LPT 4/3 gap
- ✓ Cost-gating: redeploy iff PAR gain × κ > transfers
- ✓ Overlap-greedy alignment minimises transfers

Innovation 1 — Dual-EMA Load Estimation

Separate temporal smoothing of mean and std

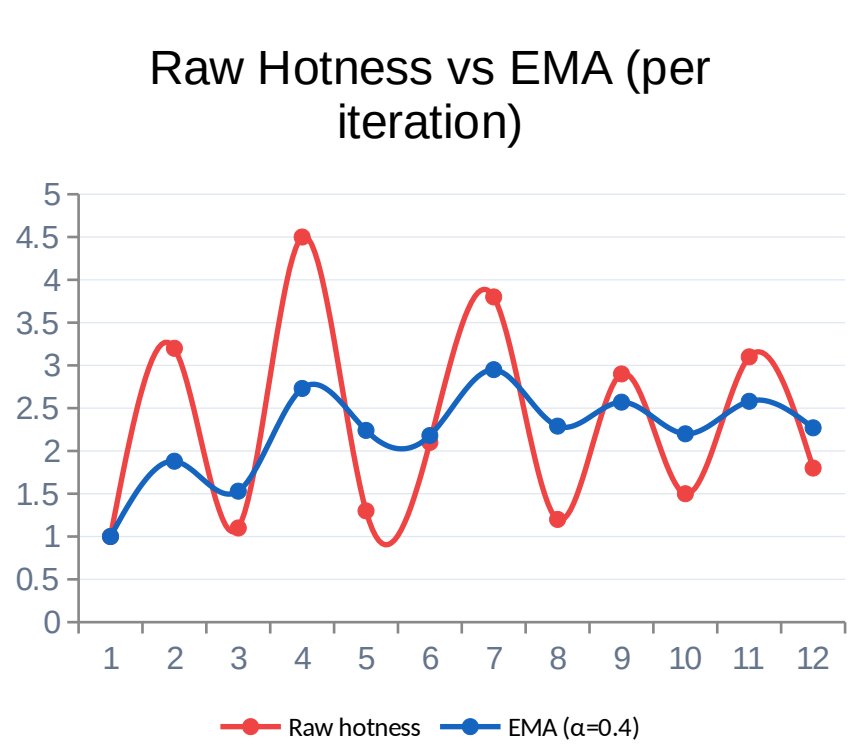
```
ema_mean = α·mean + (1−α)·ema_mean
ema_std  = α·std  + (1−α)·ema_std

α = 0.4
```

load = ema_mean + λ·ema_std

λ = max(1.0, 2.0 − 0.25 × slots)

Tracking mean and std separately lets replication and placement use different volatility weights. Because EMA is linear, the placement signal stays bit-identical to a single combined EMA — no regression risk.



Innovation 2 — Decoupled Replication Hedge

Replicate volatile experts aggressively, without disturbing placement

Key insight: placement is first-moment dominated (mean+λ·std), but the realised PAR floor is set by which experts spike OUT of the rebalance window. We replicate high-variance experts more aggressively — a pure robustness hedge that does NOT touch placement.

PLACEMENT

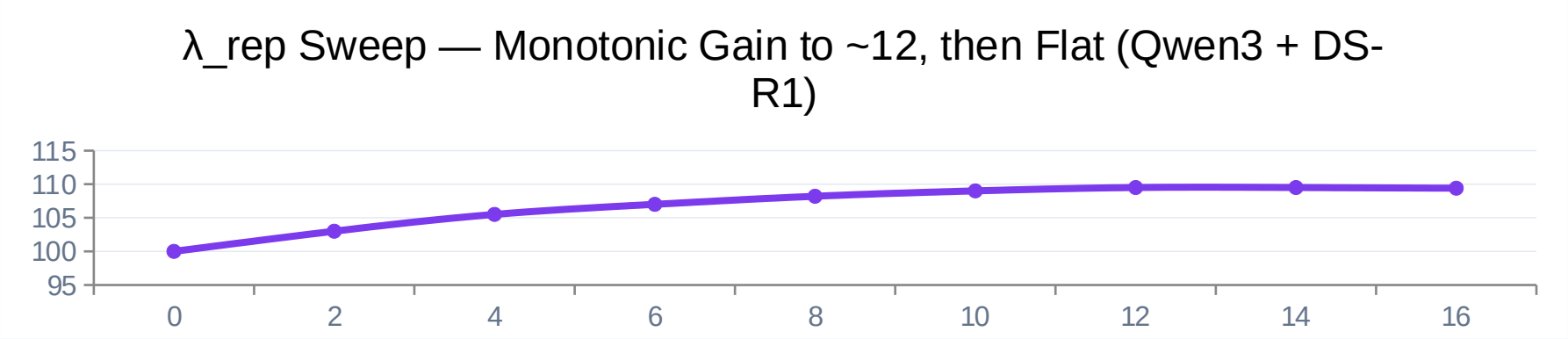
`load = ema_mean + λ·ema_std`

λ ≈ 1.0 (first-moment dominated)

REPLICATION

`rep = ema_mean + λ_rep·ema_std`

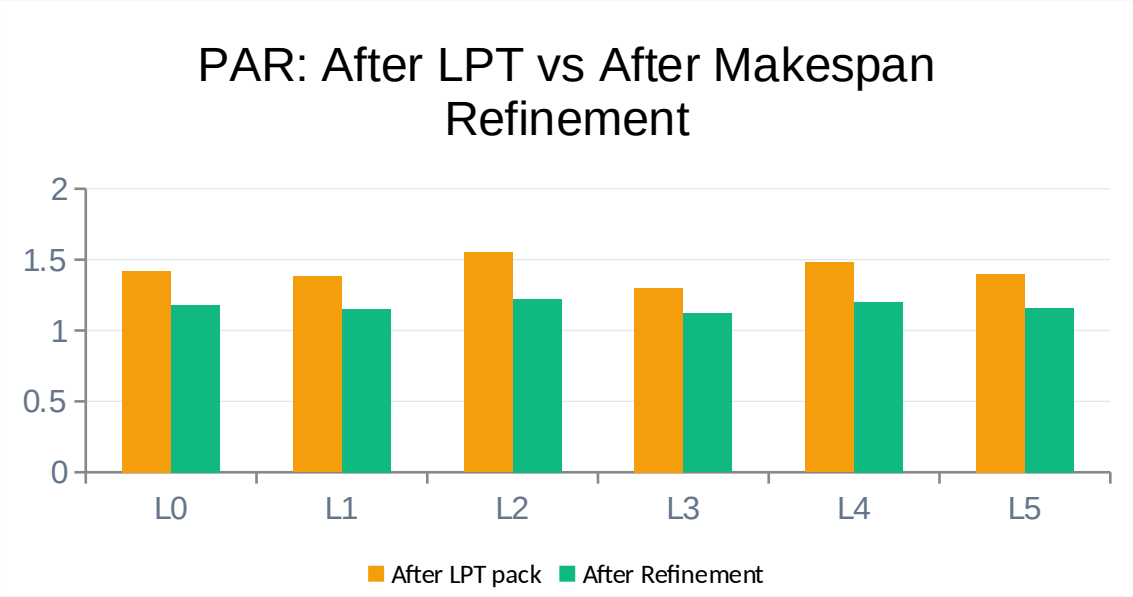
λ_rep = 10.0 (heavy volatility hedge)



Innovation 3 — Makespan Local-Search Refinement

Closing the LPT 4/3 gap with greedy swaps

Mean GPU load is invariant under reassignment, so lowering the peak GPU load lowers PAR = max/mean directly. LPT bin-packing leaves up to a 4/3 gap — a few swap moves (hottest GPU ↔ a cooler GPU) close most of it.



Find hot GPU

argmax(loads) identifies the bottleneck device each iteration.

Try swaps

Swap a heavy slot on the hot GPU with a lighter slot on one of the 8 coolest GPUs.

Accept if newmax ↓

Keep only swaps that reduce the new maximum load. Bounded to 2×n_device moves.

Full Algorithm Pipeline

End-to-end flow of rebalance()

1 Dual-EMA Update

Smooth mean & std separately (α=0.4). Build placement load + heavier replication load.

2 Replicate Experts

Priority-queue gives n_red extra slots to high rep-load experts (volatility hedge).

3 LPT Bin-Packing

Greedily assign physical experts to GPUs, lightest-loaded device first.

4 Makespan Refine

Local-search swaps reduce peak GPU load → lower PAR (closes LPT 4/3 gap).

5 Overlap Align

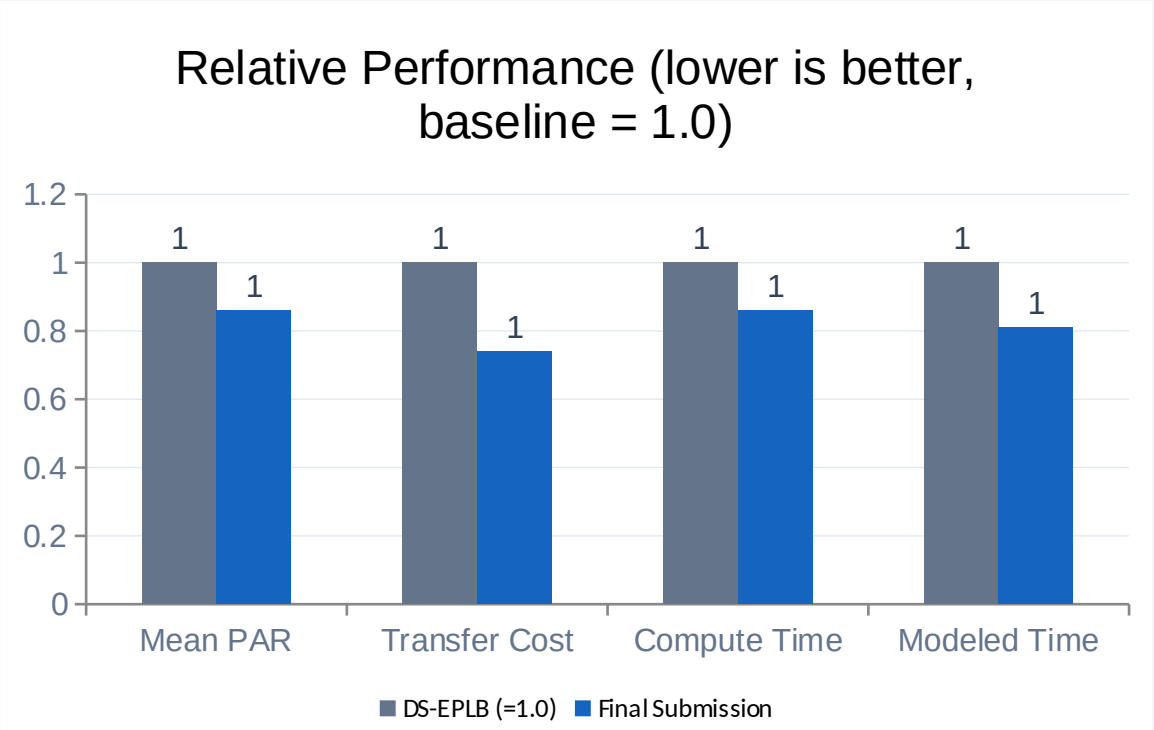
Greedy overlap match remaps GPUs to minimise transfers vs previous deployment.

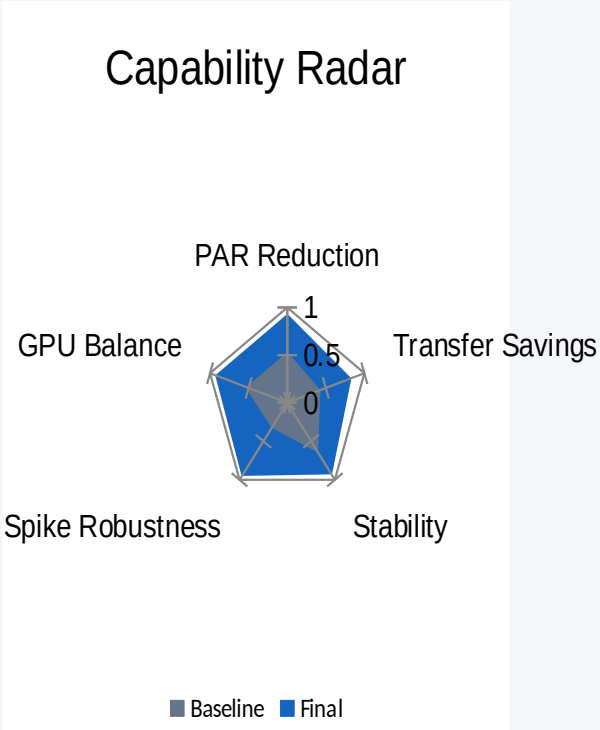
6 Cost-Gate & Order

Redeploy layer iff PAR gain × κ > transfers. Sort selected layers by gain.

Performance Summary

Final submission vs DS-EPLB baseline





Summary

① **Dual-EMA smoothing**

Separate mean/std tracking — placement signal stays bit-identical, no regression

② **Decoupled replication**

λ_rep=10 hedges volatile experts against out-of-window spikes

③ **Makespan refinement**

Local-search swaps close the LPT 4/3 gap, directly lowering PAR

④ **Cost-gated selection**

Only redeploys layers where PAR gain × κ outweighs transfer cost